SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 6 OF 10

Redis & Kafka

12 Questions | Caching Strategies | Distributed Locks | Streams | Kafka Internals | EDA

Redis & Kafka

PART 1 — REDIS

Q1: Why Redis? What makes it fast and when should you NOT use it?

Redis (Remote Dictionary Server) is an in-memory data structure store. Its speed comes from a combination of architectural decisions that eliminate every source of latency conventional databases have.

Why it's fast: Everything lives in RAM — no disk IO for reads or writes. Single-threaded command execution — no lock contention, no context switching overhead between threads. Simple data structures with O(1) or O(log N) operations. Non-blocking IO using an event loop (similar to asyncio). Network protocol (RESP) is intentionally simple to parse.

Single-threaded reality: Redis 6+ introduced I/O threading for network read/write, but command execution is still single-threaded. This means one slow command (like KEYS * on a million-key keyspace) blocks every other client. Avoid KEYS in production — use SCAN instead.

When NOT to use Redis: As your primary database — Redis is not durable by default (data lost on restart without persistence config). For large blobs — Redis is memory-constrained; storing 10MB JSON objects defeats the purpose. For complex relational queries — Redis has no joins or SQL. For data that must survive restarts with zero loss — use PostgreSQL with Redis as a cache in front.

```
# Redis data structure commands — all O(1) unless noted
SET user:42:name 'Gautham'          # String
GET user:42:name                     # -> 'Gautham'
INCR user:42:login_count             # Atomic increment
HSET user:42 name 'Gautham' age 22   # Hash (object)
HGETALL user:42                      # -> {name: Gautham, age: 22}
LPUSH notifications:42 'msg1'        # List (queue/stack)
SADD online_users 42 87 99           # Set (unique members)
ZADD leaderboard 1500 'user:42'      # Sorted Set (ranked)
ZRANGE leaderboard 0 9 WITHSCORES REV  # Top 10 — O(log N + K)
```

Q2: Caching strategies — Cache-Aside, Write-Through, Write-Behind, Read-Through.

Choosing the wrong caching strategy is one of the most common sources of stale data and cache inconsistency bugs in production systems.

Cache-Aside (Lazy Loading): Application checks cache first. On miss, fetches from DB, stores in cache, returns. Most common pattern. Cache only contains what's actually been requested. Downside: first request after expiry is slow (cache miss penalty). Risk of stale data if cache TTL is too long.

```
# Cache-Aside pattern
async def get_user(user_id: int):
    cache_key = f'user:{user_id}'

    # 1. Check cache
    cached = await redis.get(cache_key)
    if cached:
        return json.loads(cached)   # cache hit

    # 2. Cache miss — fetch from DB
    user = await db.fetch_user(user_id)
    if not user:
        # Cache negative results too — prevents DB hammering
        await redis.setex(cache_key, 30, 'null')  # short TTL for nulls
        return None
```

```
    # 3. Store in cache with TTL
    await redis.setex(cache_key, 3600, json.dumps(user))
    return user

# On update — invalidate cache
async def update_user(user_id: int, data: dict):
    await db.update_user(user_id, data)
    await redis.delete(f'user:{user_id}')   # invalidate
```

Write-Through: Every write goes to cache AND DB synchronously. Cache is always consistent. No stale reads. Downside: write latency doubles (must write both). Cache fills with data that may never be read (wastes memory).

Write-Behind (Write-Back): Writes go to cache immediately; DB write is async (batched). Lowest write latency. Risk: data loss if cache fails before DB write. Use for high-write workloads where some loss is acceptable (analytics counters, activity tracking).

Read-Through: Cache sits in front of DB as a library/proxy. Application only talks to cache; cache handles DB interaction on miss. Cleaner application code. Redis doesn't natively support this — needs a caching layer like Momento or custom middleware.

Q3: Cache invalidation — the hardest problem in caching.

'There are only two hard things in Computer Science: cache invalidation and naming things.' Cache invalidation is hard because you must decide: when is cached data stale? How do you know when to invalidate? What if multiple services cache the same data?

TTL-based expiry: Set a Time-To-Live. Simple, predictable, no coordination needed. Tradeoff: stale window = TTL duration. Choose TTL based on acceptable staleness. User profiles: 1 hour. Product prices: 5 minutes. Stock quotes: 1 second.

Event-driven invalidation: Publish an invalidation event when data changes. All cache holders subscribe and delete their cached copies. More complex but more consistent. Implement via Redis pub/sub, Kafka, or a CDN purge API.

Cache stampede / thundering herd: When a popular cache key expires, hundreds of concurrent requests all get a cache miss simultaneously — all hit the DB at once, overwhelming it. Solutions: (1) Probabilistic early expiration — randomly expire before TTL to pre-warm. (2) Mutex/lock on cache miss — only one request rebuilds, others wait. (3) Stale-while-revalidate — serve stale data while one background process refreshes it.

```
# Mutex pattern to prevent stampede
async def get_with_lock(key: str, rebuild_fn, ttl: int):
    # Try cache first
    val = await redis.get(key)
    if val: return json.loads(val)

    # Acquire distributed lock
    lock_key = f'lock:{key}'
    lock = await redis.set(lock_key, '1', nx=True, ex=10)  # NX = only if not exists

    if lock:
        # We got the lock — rebuild the cache
        try:
            val = await rebuild_fn()
            await redis.setex(key, ttl, json.dumps(val))
            return val
        finally:
            await redis.delete(lock_key)
    else:
        # Another process is rebuilding — wait and retry
        await asyncio.sleep(0.1)
        return await redis.get(key)  # should be populated now
```

Cache key design: Keys should be predictable and namespaced: user:{id}, user:{id}:orders, product:{id}:v2. Use version suffixes when changing cached data shape — lets you deploy without invalidating all at once.

Q4: Redis persistence — RDB vs AOF, and when each matters.

By default, Redis is volatile — a restart loses all data. Persistence modes let you control the durability/performance tradeoff.

RDB (Redis Database Snapshots): Periodic point-in-time snapshots written to disk (dump.rdb). Configured as: 'save 900 1' (snapshot if 1 change in 900s). Fast restarts (load entire snapshot). Risk: up to the snapshot interval of data loss on crash. Good for: cache-only use cases where some loss is acceptable.

AOF (Append-Only File): Every write command is appended to a log file. On restart, Redis replays the log. Three fsync policies: always (max durability, slowest), everysec (1 second max loss, default), no (OS decides — fastest, least durable). Good for: data that must survive restarts.

AOF rewriting: The AOF grows indefinitely. Redis periodically rewrites it — compacting history into the minimum set of commands that reproduce current state. Happens in a background fork — no downtime.

Best practice: Use both: RDB for fast restarts and point-in-time backups; AOF for durability. Redis Cluster and Redis Sentinel add replication and failover on top of persistence. For session storage and distributed locks, always enable at least AOF with everysec.

```
# redis.conf — recommended production persistence
save 3600 1      # RDB: snapshot if 1+ changes in last hour
save 300 100     # RDB: snapshot if 100+ changes in 5 min
save 60 10000    # RDB: snapshot if 10000+ changes in 1 min

appendonly yes
appendfsync everysec    # AOF: max 1 second data loss
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
```

Q5: Distributed locks with Redis — SETNX, Redlock, and pitfalls.

A distributed lock ensures only one process/node executes a critical section at a time across a distributed system. Redis is commonly used for this — but naively implementing it has subtle failure modes.

Basic pattern (SETNX + EXPIRE): SET key value NX EX seconds — atomic set-if-not-exists with expiry. NX ensures only one caller gets the lock. EX ensures the lock auto-releases if the holder crashes (prevents deadlock).

```
import uuid, asyncio
from redis.asyncio import Redis

class DistributedLock:
    def __init__(self, redis: Redis, key: str, ttl: int = 30):
        self.redis = redis
        self.key = f'lock:{key}'
        self.ttl = ttl
        self.token = str(uuid.uuid4())  # unique per lock holder

    async def acquire(self) -> bool:
        # SET key token NX EX ttl — atomic
        result = await self.redis.set(
            self.key, self.token, nx=True, ex=self.ttl
        )
        return result is not None
```

```
    async def release(self):
        # Lua script: only delete if WE own the lock
        # Prevents releasing another holder's lock
        script = '''
        if redis.call('get', KEYS[1]) == ARGV[1] then
            return redis.call('del', KEYS[1])
        else return 0 end
        '''
        await self.redis.eval(script, 1, self.key, self.token)

    async def __aenter__(self):
        acquired = await self.acquire()
        if not acquired:
            raise RuntimeError('Could not acquire lock')
        return self

    async def __aexit__(self, *args):
        await self.release()

# Usage
async with DistributedLock(redis, 'payment:user:42'):
    await process_payment(user_id=42)
```

The unique token is critical: Without a unique token per lock acquisition, a process could release a lock it no longer owns. Scenario: Process A acquires lock, gets delayed (GC pause), lock expires, Process B acquires it, Process A resumes and releases B's lock. The token prevents this.

Redlock algorithm: For high-reliability distributed locks across Redis Cluster: acquire lock on N/2+1 independent Redis nodes. The lock is valid only if acquired on majority nodes within the lock TTL. More complex — use the redlock-py library. For most use cases, single-node SETNX with proper TTL is sufficient.

Clock drift warning: Distributed locks based on TTL assume clocks are roughly synchronized. NTP drift across nodes can cause a lock to expire on one node while another thinks it's still valid. Design critical sections to be idempotent so double-execution is safe, regardless of lock guarantees.

Q6: Redis pub/sub vs Streams — which to use when?

Redis offers two messaging models. Pub/Sub is fire-and-forget; Streams are a persistent, consumer-group-aware message log. The choice depends on whether you need message persistence and delivery guarantees.

Pub/Sub: Publisher sends to a channel; all current subscribers receive it. No persistence — if a subscriber is offline, the message is lost. No acknowledgment. No consumer groups. Good for: real-time notifications, live dashboards, cache invalidation signals, WebSocket fan-out.

Redis Streams (XADD/XREAD): A persistent, append-only log. Messages survive subscriber downtime. Consumer groups allow multiple workers to share a stream (each message delivered to one worker in the group). Acknowledgment model (XACK). Pending entries list tracks unacknowledged messages. Good for: event sourcing, task queues, audit logs, anything where message loss is unacceptable.

```
# Pub/Sub — simple broadcast
# Publisher:
await redis.publish('cache:invalidate', json.dumps({'key': 'user:42'}))

# Subscriber:
async with redis.pubsub() as ps:
    await ps.subscribe('cache:invalidate')
    async for message in ps.listen():
        if message['type'] == 'message':
            data = json.loads(message['data'])
            await local_cache.delete(data['key'])

# Streams — durable event log
# Producer:
```

```
await redis.xadd('orders', {
    'order_id': '42',
    'user_id': '7',
    'amount': '99.99',
})

# Consumer group — distributes messages across workers
await redis.xgroup_create('orders', 'order-processors', id='0', mkstream=True)

# Worker:
messages = await redis.xreadgroup(
    'order-processors', 'worker-1',
    {'orders': '>'},   # '>' means new, undelivered messages
    count=10, block=5000
)
for stream, msgs in messages:
    for msg_id, data in msgs:
        await process_order(data)
        await redis.xack('orders', 'order-processors', msg_id)
```

PART 2 — KAFKA

Q7: Why Kafka? Core concepts: topics, partitions, offsets, consumer groups.

Apache Kafka is a distributed, partitioned, replicated commit log. It is fundamentally different from a message queue — it retains messages after consumption, allows replaying, and scales to millions of events per second.

Topic: A named, ordered, immutable log of events. Think of it as a database table for events. Producers append to topics; consumers read from them. Topics are divided into partitions for parallelism.

Partition: The unit of parallelism and ordering. Each partition is an ordered, immutable sequence of messages. Messages within a partition are strictly ordered. Ordering across partitions is not guaranteed. A topic with 12 partitions can be consumed by up to 12 consumers in parallel.

Offset: Every message in a partition has a monotonically increasing integer offset. Consumers track their position by offset. This is how Kafka enables replay — a consumer can seek to offset 0 and re-read the entire history.

Consumer Group: A group of consumers that jointly consume a topic. Each partition is assigned to exactly one consumer in the group. Add more consumers to scale consumption. If consumers > partitions, some consumers are idle. Different consumer groups are completely independent — they each maintain their own offsets.

```
# Topic: orders — 6 partitions
# Consumer Group: payment-service — 3 consumers
#
# Partition 0 -> Consumer A
# Partition 1 -> Consumer A
# Partition 2 -> Consumer B
# Partition 3 -> Consumer B
# Partition 4 -> Consumer C
# Partition 5 -> Consumer C
#
# Consumer Group: analytics-service — 1 consumer
# Partition 0,1,2,3,4,5 -> Consumer X (all partitions)
# analytics reads all events independently of payment-service

# Python producer
from confluent_kafka import Producer
p = Producer({'bootstrap.servers': 'kafka:9092'})
p.produce(
    topic='orders',
    key=str(order.user_id).encode(),   # same user -> same partition
    value=json.dumps(order.dict()).encode(),
```

```
    callback=delivery_report
)
p.flush()
```

Q8: Message ordering in Kafka — how to guarantee it.

Kafka guarantees ordering within a partition, not across partitions. If you need messages for a given entity to be processed in order, you must ensure they go to the same partition.

Partition key: When producing, specify a key. Kafka hashes the key to determine the partition. All messages with the same key go to the same partition — therefore arrive in order. Use user_id, order_id, or entity_id as the key.

Single partition topics: A topic with 1 partition guarantees global ordering but sacrifices parallelism. Only one consumer can read it. Use only when global ordering is truly required and throughput is low.

Ordering within consumer: Even with the right partition key, a consumer processing messages concurrently (goroutine pool) can process out of order. Either process sequentially or use a sequence number in the message payload and sort/buffer before processing.

```
# Guaranteed ordering for a user's events
p.produce(
    topic='user-events',
    key=f'user:{user_id}'.encode(),   # consistent key = consistent partition
    value=event_payload,
)

# ALL events for user 42 go to the same partition
# Consumer processes them in order: create -> update -> delete

# Anti-pattern: random partition key
p.produce(topic='user-events', value=event_payload)
# No key = round-robin partitioning = no ordering guarantee
```

Q9: Delivery semantics — at-most-once, at-least-once, exactly-once.

The delivery guarantee is one of the most important architectural decisions in event-driven systems. Exactly-once is seductive but comes with significant complexity and performance cost.

At-most-once: Message delivered 0 or 1 times. Commit offset before processing. If consumer crashes after commit but before processing, message is lost. Fastest. Use for: metrics, analytics where some loss is acceptable.

At-least-once: Message delivered 1 or more times. Process first, commit offset after. If consumer crashes after processing but before committing, message is redelivered and processed again. Most common. Requires idempotent consumers.

Exactly-once: Message delivered exactly once. Kafka's transactional API enables this but requires: idempotent producers (enable.idempotence=true), Kafka transactions (wrapping produce + offset commit in a transaction), and compatible consumer config (isolation.level=read_committed). Performance cost is real — use only when duplicates have business-critical consequences (payment processing).

```
# At-least-once consumer (most common pattern)
consumer = Consumer({
    'bootstrap.servers': 'kafka:9092',
    'group.id': 'payment-service',
    'auto.offset.reset': 'earliest',
    'enable.auto.commit': False,   # manual commit for at-least-once
})
consumer.subscribe(['orders'])
```

```
while True:
    msg = consumer.poll(timeout=1.0)
    if msg is None: continue

    try:
        order = json.loads(msg.value())
        # Idempotent processing: check if already processed
        if not await db.is_processed(order['id']):
            await process_order(order)
            await db.mark_processed(order['id'])
        # Only commit after successful processing
        consumer.commit(msg)
    except Exception as e:
        log.error(f'Failed processing: {e}')
        # Do NOT commit — message will be redelivered
```

Idempotency key pattern: The practical solution for at-least-once systems: include a unique event_id in every message. Before processing, check if event_id already exists in a processed_events table. If yes, skip. This makes at-least-once behave like exactly-once at the application level, without Kafka transaction overhead.

Q10: Kafka vs RabbitMQ — what are the real architectural differences?

Kafka and RabbitMQ are both messaging systems but they are designed around fundamentally different models. Choosing the wrong one causes painful migrations later.

Kafka: log-based: Messages are stored durably and retained after consumption (configurable retention — days, weeks, forever). Consumers pull at their own pace. Replay is built-in. Designed for high-throughput, ordered event streams. Consumers are 'dumb' — Kafka doesn't know or care if they've processed a message.

RabbitMQ: queue-based: Messages are deleted after acknowledgment. Push-based delivery to consumers. Rich routing via exchanges (direct, topic, fanout, headers). Better for task queues where each message is a job to be done exactly once. Broker is 'smart' — tracks delivery, handles DLQs, supports priorities.

Throughput: Kafka: millions of messages/second per broker via sequential disk writes (actually faster than random memory writes for large volumes). RabbitMQ: tens of thousands per second — sufficient for most task queues but not event streams at scale.

Replay: Kafka: built-in, just seek to an offset. This is Kafka's killer feature — replay all events to rebuild a derived database, debug a bug, or onboard a new service. RabbitMQ: no replay — once consumed and acked, message is gone.

When to use Kafka: Event streaming, event sourcing, real-time analytics pipelines, audit logs, inter-microservice event bus at scale, change data capture (CDC). If you need replay or multiple independent consumers reading the same events.

When to use RabbitMQ: Task queues (send email, resize image), job distribution to workers, request-reply patterns, complex routing logic, when messages must be consumed exactly once and deleted. Simpler operations and setup than Kafka.

```
# Kafka: same events, multiple independent consumers
Topic: user-registered
  -> Consumer Group: email-service     (sends welcome email)
  -> Consumer Group: analytics-service (updates dashboards)
  -> Consumer Group: crm-service       (creates CRM record)
# All three read independently, each maintaining their own offset
# New service added? It reads from offset 0 — gets full history

# RabbitMQ: work queue — one worker processes each job
Queue: send-email
  -> Worker 1 | Worker 2 | Worker 3
# Each email job delivered to exactly one worker
# Job deleted after ack — no replay
```

Q11: How does Kafka scale? Replication, leaders, and consumer lag.

Kafka's scaling model is core to understanding why it handles massive throughput reliably. Every concept — replication factor, ISR, consumer lag — maps to a production concern you'll deal with.

Replication: Each partition has a replication factor (typically 3). One broker is the leader for each partition; others are followers (ISR — In-Sync Replicas). Producers write to the leader; followers replicate. If the leader fails, a follower is elected leader — zero data loss if replication factor >= 2.

Producer acks: acks=0: fire-and-forget. acks=1: leader acknowledges (follower failure could lose the message). acks=all (or -1): all ISR replicas acknowledge — strongest durability guarantee, highest latency.

Consumer lag: The difference between the latest offset produced and the offset committed by a consumer. High lag = consumers falling behind producers. Monitor via kafka-consumer-groups.sh or Prometheus + Kafka Exporter. Remedies: add consumers (up to partition count), optimize consumer processing, or scale partitions.

Partition count planning: Cannot decrease partitions after creation (only increase). Key-based ordering breaks when you add partitions (the hash changes). Plan partition count for 2-3x your expected peak consumer count. More partitions = more parallelism but more leader election overhead and file handles.

```
# Check consumer lag
kafka-consumer-groups.sh --bootstrap-server kafka:9092 \
  --group payment-service --describe

# GROUP           TOPIC    PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
# payment-service orders   0          1000            1050            50
# payment-service orders   1          2000            2200            200  <- growing!

# Lag growing on partition 1 — add a consumer to the group
# or investigate slow processing in that consumer

# Producer config for durability
producer_config = {
    'bootstrap.servers': 'kafka:9092',
    'acks': 'all',                  # wait for all ISR
    'enable.idempotence': True,     # exactly-once producer
    'retries': 10,
    'retry.backoff.ms': 100,
    'max.in.flight.requests.per.connection': 5,  # with idempotence
}
```

Q12: Event-driven architecture — patterns, benefits, and real tradeoffs.

Event-driven architecture (EDA) is a paradigm where services communicate by producing and consuming events rather than direct synchronous calls. It enables loose coupling and independent scalability — but introduces new classes of problems.

Event sourcing: Instead of storing current state, store the sequence of events that led to that state. Current state is a derived projection. Replay events to rebuild state, create new projections, or debug historical behavior. Audit log is built-in. Downside: complex queries, eventual consistency, projection rebuilds take time.

CQRS (Command Query Responsibility Segregation): Separate write model (commands that produce events) from read model (projections optimized for queries). Write side: normalized, event-sourced. Read side: denormalized, optimized for specific queries. Common with event sourcing.

Saga pattern: Manage distributed transactions across microservices using a sequence of events. Each step publishes a success or failure event. Compensating transactions undo completed steps on failure. Choreography (each service reacts to events) vs Orchestration (a central coordinator sends commands).

```
# Saga: order placement across services
# Choreography style:
```

```
# 1. OrderService publishes: order.created {order_id, user_id, items}
# 2. InventoryService listens:
#    - Reserves stock -> publishes inventory.reserved
#    - Cannot reserve -> publishes inventory.reservation.failed
# 3. PaymentService listens to inventory.reserved:
#    - Charges card -> publishes payment.completed
#    - Charge fails -> publishes payment.failed
# 4. OrderService listens:
#    - payment.completed -> confirms order, publishes order.confirmed
#    - payment.failed -> cancels order, publishes order.cancelled
# 5. InventoryService listens to order.cancelled:
#    - Releases reserved stock (compensating transaction)
```

Tradeoffs vs synchronous calls: Benefits: services are decoupled, independently deployable, naturally buffered (Kafka absorbs load spikes). Costs: eventual consistency (not immediately consistent), harder to debug (distributed traces needed), harder to reason about flow (events are implicit), at-least-once delivery needs idempotency everywhere.

Outbox pattern: Solves the dual-write problem: how do you atomically update the DB AND publish an event? Write the event to an outbox table in the same DB transaction. A separate process reads the outbox and publishes to Kafka. Guarantees no events are lost even if Kafka is temporarily unavailable.
